

Text To Audio

The screenshot shows a REST client interface with the following details:

- URL:** `https://api.groq.com/openai/v1/audio/speech`
- Method:** POST
- Body:**

```
{  "model": "canopylabs/orpheus-v1-english",  "input": "[Cheerful] vanakkam. I am a language model created by Groq. I am here to help you with any questions you may have. How can I assist you today?",  "voice": "hannah",  "response_format": "wav"}
```
- Response:** 200 OK, 1.32 s, 387.08 KB. The response body is a large audio file represented by a waveform.

Multilingual

The screenshot shows a REST client interface with the following details:

- URL:** `https://api.groq.com/openai/v1/chat/completions`
- Method:** POST
- Body:**

```
{  "role": "system",  "content": "You are an API Specialist. Always respond in the same language as the user.",  "role": "user",  "content": "വളരെ നന്ദി! ഈ API എങ്ങനെ ഉപയോഗിക്കാമെന്ന് പറയൂ."}
```
- Response:** 200 OK, 3.85 s, 1.59 KB. The response body is a JSON object containing the chat completion.

Generate API Testcases from scratch

The screenshot displays a REST client interface with a POST request to `https://api.groq.com/openai/v1/chat/completions`. The request body is in raw format, containing a system prompt and a user request. The response is a 200 OK status with a JSON body.

Request:

```
POST https://api.groq.com/openai/v1/chat/completions
```

Request Body (raw):

```
ONLY Java Rest Assured code.\n2) Do NOT include explanations, markdown, or extra text.\n3) Use TestNG as the test runner.\n4) Use meaningful class, method, and variable names.\n5) Include imports required for Rest Assured, TestNG, and assertions.\n6) Use reusable base URI setup.\n7) Cover CRUD operations for Pet resource.\n8) Add assertions for status code and important response fields.\n9) Keep the code clean, executable, and production-style.\n10) Do NOT use placeholder comments.\n\nC - CONTEXT\nThe code will be used by QA engineers for API automation practice using Swagger Petstore.\n\nP - PERSONA\nYou are a Senior API Test Automation Architect creating enterprise-style reusable Rest Assured automation code.\n\nO - OUTPUT\nReturn ONLY executable Java code with:\n- One test class\n- @BeforeClass setup\n- Test methods for createPet, getPetById, updatePet, deletePet\n- Request body creation\n- Response validation\n\nRules:\n- No markdown fences\n- No explanation text\n- No assumptions outside Swagger Petstore Pet API\n- 
```

Response:

```
{
  "id": "chatcmpl-75b8789d-f32d-4c68-af80-e2be5de50ab5",
  "object": "chat.completion",
  "created": 1784281842,
  "model": "openai/gpt-oss-120b",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "import io.restassured.RestAssured;\nimport io.restassured.http.ContentType;\nimport io.restassured.response.Response;\nimport org.testng.Assert;\nimport org.testng.annotations.BeforeClass;\nimport org.testng.annotations.Test;\nimport java.util.HashMap;\nimport java.util.Map;\nimport java.util.Random;\n\npublic class PetApiTests {\n\n    private static final String BASE_URI = \"https://\"
```